

AI & Assisted coding

Assignment-16.1

Ht no.:2303A51690

Batch :28

Task1: Student Information System Schema

PROMPT:

generate SQL create table statement for a student information system that includes tables students, courses and enrollments also define primary keys,foreign keys and relationships between the tables

generate sql queries to insert sample data into the students, courses and enrollments tables

generate sql query to insert a new student record

generate sql query to fetch all courses enrolled by a student

CODE:

```
1ad1b.sql
1  -- generate SQL create table statement for a student information system that includes tables students, courses and enrollments
2  --also define primary keys,foreign keys and relationships between the tables
3  CREATE TABLE students (
4      student_id INT PRIMARY KEY,
5      first_name VARCHAR(50) NOT NULL,
6      last_name VARCHAR(50) NOT NULL,
7      email VARCHAR(100) UNIQUE NOT NULL
8  );
9
10 CREATE TABLE courses (
11     course_id INT PRIMARY KEY,
12     course_name VARCHAR(100) NOT NULL,
13     credits INT NOT NULL
14 );
15
16 CREATE TABLE enrollments (
17     student_id INT,
18     course_id INT,
19     PRIMARY KEY (student_id, course_id),
20     FOREIGN KEY (student_id) REFERENCES students(student_id),
21     FOREIGN KEY (course_id) REFERENCES courses(course_id)
22 );
23
24 --generate sql queries to insert sample data into the students, courses and enrollments tables
25 INSERT INTO students (student_id, first_name, last_name, email) VALUES
26 (1, 'John', 'Doe', 'john.doe@example.com'),
27 (2, 'Jane', 'Smith', 'jane.smith@example.com');
28
29 INSERT INTO courses (course_id, course_name, credits) VALUES
30 (101, 'Introduction to Computer Science', 3),
31 (102, 'Calculus I', 4);
32
33 INSERT INTO enrollments (student_id, course_id) VALUES
34 (1, 101),
35 (1, 102),
36 (2, 101);
37
```

```

27 (2, 'Jane', 'Smith', 'jane.smith@example.com');
28
29 INSERT INTO courses (course_id, course_name, credits) VALUES
30 (101, 'Introduction to Computer Science', 3),
31 (102, 'Calculus I', 4);
32
33 INSERT INTO enrollments (student_id, course_id) VALUES
34 (1, 101),
35 (1, 102),
36 (2, 101);
37
38 --generate sql query to insert a new student record
39 INSERT INTO students (student_id, first_name, last_name, email) VALUES
40 (3, 'Alice', 'Johnson', 'alice.johnson@example.com');
41
42 --generate sql query to fetch all courses enrolled by a student
43 SELECT c.course_name
44 FROM courses c
45 JOIN enrollments e ON c.course_id = e.course_id
46 WHERE e.student_id = 1;
47
48
49 |
50
51 --generate sql query to count number of students in each course
52 SELECT c.course_name, COUNT(e.student_id) AS student_count
53 FROM courses c
54 LEFT JOIN enrollments e ON c.course_id = e.course_id
55 GROUP BY c.course_name;

```

OUTPUT:

lab16.sql

```
1  -- generate SQL create table statements
2  --also define primary keys,foreign keys
3  CREATE TABLE students (
4      student_id INT PRIMARY KEY,
5      first_name VARCHAR(50) NOT NULL,
6      last_name VARCHAR(50) NOT NULL,
7      email VARCHAR(100) UNIQUE NOT NULL
8  );
9
10 CREATE TABLE courses (
11     course_id INT PRIMARY KEY,
12     course_name VARCHAR(100) NOT NULL,
13     credits INT NOT NULL
14 );
15
16 CREATE TABLE enrollments (
17     student_id INT,
18     course_id INT,
19     PRIMARY KEY (student_id, course_id),
20     FOREIGN KEY (student_id) REFERENCES students(student_id),
21     FOREIGN KEY (course_id) REFERENCES courses(course_id)
22 );
23
```

SQL < 1 / 1 > 1 - 0 of 0

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);
```

SQL < 1 / 1 > 1 - 0 of 0

```
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  credits INT NOT NULL
);
```

SQL < 1 / 1 > 1 - 0 of 0

```
CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES
  students(student_id),
  FOREIGN KEY (course_id) REFERENCES
  courses(course_id)
);
```

lab16.sql

```
9
10 -- CREATE TABLE courses (
11 --     course_id INT PRIMARY KEY,
12 --     course_name VARCHAR(100) NOT NULL,
13 --     credits INT NOT NULL
14 -- );
15
16 -- CREATE TABLE enrollments (
17 --     student_id INT,
18 --     course_id INT,
19 --     PRIMARY KEY (student_id, course_id),
20 --     FOREIGN KEY (student_id) REFERENCES students(student_id),
21 --     FOREIGN KEY (course_id) REFERENCES courses(course_id)
22 -- );
23
24 --generate sql queries to insert sample data into the students, courses and
25 --enrollments tables
26 INSERT INTO students (student_id, first_name, last_name, email) VALUES
27 (1, 'John', 'Doe', 'john.doe@example.com'),
28 (2, 'Jane', 'Smith', 'jane.smith@example.com');
29
30 INSERT INTO courses (course_id, course_name, credits) VALUES
31 (101, 'Introduction to Computer Science', 3),
32 (102, 'Calculus I', 4);
33
34 INSERT INTO enrollments (student_id, course_id) VALUES
35 (1, 101),
36 (1, 102),
37 (2, 101);
38
```

SQL < 1 / 1 > 1 - 0 of 0

```
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@example.com'),
(2, 'Jane', 'Smith', 'jane.smith@example.com');
```

SQL < 1 / 1 > 1 - 0 of 0

```
INSERT INTO courses (course_id, course_name, credits) VALUES
(101, 'Introduction to Computer Science', 3),
(102, 'Calculus I', 4);
```

SQL < 1 / 1 > 1 - 0 of 0

```
INSERT INTO enrollments (student_id, course_id) VALUES
(1, 101),
(1, 102),
(2, 101);
```

```
lab16.sql
32
33 -- INSERT INTO enrollments (student_id, course_id) VALUES
34 -- (1, 101),
35 -- (1, 102),
36 -- (2, 101);
37
38 --generate sql query to insert a new student record
39 INSERT INTO students (student_id, first_name, last_name, email) VALUES
40 (3, 'Alice', 'Johnson', 'alice.johnson@example.com');
41
42
```

```
SQL
1 / 1 1 - 0 of 0
INSERT INTO students (student_id, first_name, last_name, email) VALUES
(3, 'Alice', 'Johnson', 'alice.johnson@example.com');
```

```
lab16.sql
40 -- (3, 'Alice', 'Johnson', 'alice.johnson@example.com');
41
42 --generate sql query to fetch all courses enrolled by a student
43 SELECT c.course_name
44 FROM courses c
45 JOIN enrollments e ON c.course_id = e.course_id
46 WHERE e.student_id = 1;
47
48
49
50
51
52
```

```
SQL
1 / 1 1 - 2 of 2
course_name
Introduction to Computer Science
Calculus I
```

```
Labtest.py labtest2.py lab_15_1.py lab16.db lab16.sql SQLite
lab16.sql
55
56
57
58
59
60 --generate sql query to count number of students in each course
61 SELECT c.course_name, COUNT(e.student_id) AS student_count
62 FROM courses c
63 LEFT JOIN enrollments e ON c.course_id = e.course_id
64 GROUP BY c.course_name;
```

```
SQL
1 / 1 1 - 2 of 2
course_name student_count
Calculus I 1
Introduction to Computer Science 2
```

EXPLANATION:

- Designed a relational database schema with Students, Courses, and Enrollments tables using primary and foreign keys.
- Implemented relationships using a junction table (Enrollments) to handle many-to-many mapping between students and courses.
- Performed CRUD operations including inserting student/course data and retrieving enrolled courses using JOIN queries.
- Used aggregate function (COUNT with GROUP BY) to calculate the number of students in each course.

Task 2 – Hospital Management Database

PROMPT:

create a schema for hospital management system

insert sample data into all tables

query to retrieve list of all appointments for a specific doctor

query to retrieve patient history by patient Id

query to count total patients treated by each doctor

CODE:

```
lab16.sql  JS  script.js  #  styles.css  <>  index.html  L1.py  Labtest.py  labtest2.py  lab_15_1.py  lab16.db  lab16.sql  X

1  --create a schema for hospital management system
2  CREATE TABLE patients (
3      patient_id INT PRIMARY KEY,
4      first_name VARCHAR(50) NOT NULL,
5      last_name VARCHAR(50) NOT NULL,
6      date_of_birth DATE NOT NULL,
7      gender VARCHAR(10) NOT NULL,
8      phone VARCHAR(20),
9      email VARCHAR(100) UNIQUE NOT NULL
10 );
11
12 CREATE TABLE doctors (
13     doctor_id INT PRIMARY KEY,
14     first_name VARCHAR(50) NOT NULL,
15     last_name VARCHAR(50) NOT NULL,
16     specialty VARCHAR(100) NOT NULL,
17     phone VARCHAR(20),
18     email VARCHAR(100) UNIQUE NOT NULL
19 );
20
21 CREATE TABLE appointments (
22     appointment_id INT PRIMARY KEY,
23     patient_id INT,
24     doctor_id INT,
25     appointment_date DATE NOT NULL,
26     appointment_time TIME NOT NULL,
27     status VARCHAR(20) DEFAULT 'Scheduled',
28     FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
29     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
30 );
31
32
33 --insert sample data into all tables
34 INSERT INTO patients (patient_id, first_name, last_name, date_of_birth, gender, phone, email) VALUES
35 (1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@example.com'),
36 (2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.smith@example.com');
37 INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone, email) VALUES
```

```
JS script.js # styles.css index.html L1.py Labtest.py labtest2.py lab_15_1.py lab16.db lab16.sql x index1.html
lab16.sql

--insert sample data into all tables
INSERT INTO patients (patient_id, first_name, last_name, date_of_birth, gender, phone, email) VALUES
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@example.com'),
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.smith@example.com');
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone, email) VALUES
(1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210', 'dr.smith@example.com'),
(2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543', 'dr.davis@example.com');
INSERT INTO appointments (appointment_id, patient_id, doctor_id, appointment_date, appointment_time, status) VALUES
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');

--query to retrieve list of all appointments for a specific doctor
SELECT a.appointment_id, p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date, a.appointment_time, a.status
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;

--query to retrieve patient history by patient Id
SELECT p.first_name AS patient_first_name, p.last_name AS patient_last_name, a.appointment_date, a.appointment_time, a.status, d.first_name AS doctor_first_name, d.last_name AS doctor_last_name
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;

--query to count total patients treated by each doctor
SELECT d.first_name AS doctor_first_name, d.last_name AS doctor_last_name, COUNT(a.appointment_id) AS total_patients
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

OUTPUT:

```
Labtest.py labtest2.py lab_15_1.py lab16.db lab16.sql x ... SQLite x
lab16.sql

1 --create a schema for hospital management system
2 CREATE TABLE patients (
3     patient_id INT PRIMARY KEY,
4     first_name VARCHAR(50) NOT NULL,
5     last_name VARCHAR(50) NOT NULL,
6     date_of_birth DATE NOT NULL,
7     gender VARCHAR(10) NOT NULL,
8     phone VARCHAR(20),
9     email VARCHAR(100) UNIQUE NOT NULL
10 );
11
12 CREATE TABLE doctors (
13     doctor_id INT PRIMARY KEY,
14     first_name VARCHAR(50) NOT NULL,
15     last_name VARCHAR(50) NOT NULL,
16     specialty VARCHAR(100) NOT NULL,
17     phone VARCHAR(20),
18     email VARCHAR(100) UNIQUE NOT NULL
19 );
20
21 CREATE TABLE appointments (
22     appointment_id INT PRIMARY KEY,
23     patient_id INT,
24     doctor_id INT,
25     appointment_date DATE NOT NULL,
26     appointment_time TIME NOT NULL,
27     status VARCHAR(20) DEFAULT 'Scheduled',
28     FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
29     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
30 );
31

SQL v < 1 / 1 > 1 - 0 of 0
CREATE TABLE patients (
patient_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
date_of_birth DATE NOT NULL,
gender VARCHAR(10) NOT NULL,
phone VARCHAR(20),
email VARCHAR(100) UNIQUE NOT NULL
);

SQL v < 1 / 1 > 1 - 0 of 0
CREATE TABLE doctors (
doctor_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
specialty VARCHAR(100) NOT NULL,
phone VARCHAR(20),
email VARCHAR(100) UNIQUE NOT NULL
);

SQL v < 1 / 1 > 1 - 0 of 0
CREATE TABLE appointments (
appointment_id INT PRIMARY KEY,
patient_id INT,
doctor_id INT,
appointment_date DATE NOT NULL,
appointment_time TIME NOT NULL,
status VARCHAR(20) DEFAULT 'Scheduled',
FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

lab16.sql

```
21 CREATE TABLE appointments (  
28     FOREIGN KEY (patient_id) REFERENCES patients(patient_id),  
29     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)  
30 );  
31  
32  
33 --insert sample data into all tables  
34 INSERT INTO patients (patient_id, first_name, last_name, date_of_birth, gen  
35 (1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890', 'john.doe@example.  
36 (2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567', 'jane.smith@ex  
37 INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone, em  
38 (1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210', 'dr.smith@exampl  
39 (2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543', 'dr.davis@exampl  
40 INSERT INTO appointments (appointment_id, patient_id, doctor_id, appointme  
41 (1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),  
42 (2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');  
43  
44
```

SQL ▾

< 1 / 1 > 1 - 0 of 0

```
INSERT INTO patients (patient_id, first_name, last_name, date_of_birth,  
gender, phone, email) VALUES  
(1, 'John', 'Doe', '1980-01-01', 'Male', '123-456-7890',  
'john.doe@example.com'),  
(2, 'Jane', 'Smith', '1990-02-15', 'Female', '555-123-4567',  
'jane.smith@example.com');
```

SQL ▾

< 1 / 1 > 1 - 0 of 0

```
INSERT INTO doctors (doctor_id, first_name, last_name, specialty, phone,  
email) VALUES  
(1, 'Dr. Smith', 'Johnson', 'Cardiology', '987-654-3210',  
'dr.smith@example.com'),  
(2, 'Dr. Emily', 'Davis', 'Pediatrics', '555-987-6543',  
'dr.davis@example.com');
```

SQL ▾

< 1 / 1 > 1 - 0 of 0

```
INSERT INTO appointments (appointment_id, patient_id, doctor_id,  
appointment_date, appointment_time, status) VALUES  
(1, 1, 1, '2024-07-01', '10:00:00', 'Scheduled'),  
(2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');
```

lab16.sql

```
1 SELECT a.appointment_id, p.first_name AS patient_first_name, p.last  
2 (2, 2, 2, '2024-07-02', '14:00:00', 'Scheduled');  
3  
4 --query to retrieve list of all appointments for a specific doctor  
5 SELECT a.appointment_id, p.first_name AS patient_first_name, p.last  
6 FROM appointments a  
7 JOIN patients p ON a.patient_id = p.patient_id  
8 WHERE a.doctor_id = 1;  
9  
0  
1  
2  
3  
4
```

SQL ▾

< 1 / 1 > 1 - 1 of 1

appointment_id	patient_first_name	patient_last_name	appointment_date	appointm
1	John	Doe	2024-07-01	10:00:00

lab16.sql

```
55  
56  
57  
58  
59  
60 --query to retrieve patient history by patient id  
61 SELECT p.first_name AS patient_first_name, p.last_name AS patient_last_name  
62 FROM patients p  
63 JOIN appointments a ON p.patient_id = a.patient_id  
64 JOIN doctors d ON a.doctor_id = d.doctor_id  
65 WHERE p.patient_id = 1;  
66  
67  
68  
69  
70
```

SQL ▾

< 1 / 1 > 1 - 1 of 1

patient_first_name	patient_last_name	appointment_date	appointment_time
John	Doe	2024-07-01	10:00:00

lab16.sql

```
69  
70  
71  
72  
73 --query to count total patients treated by each doctor  
74 SELECT d.first_name AS doctor_first_name, d.last_name AS doctor_last_name,  
75 .appointment_id AS total_patients  
76 FROM doctors d  
77 LEFT JOIN appointments a ON d.doctor_id = a.doctor_id  
78 GROUP BY d.doctor_id;  
79
```

SQL ▾

< 1 / 1 > 1 - 2 of 2

doctor_first_name	doctor_last_name	total_patients
Dr. Smith	Johnson	1
Dr. Emily	Davis	1

EXPLANATION:

- Designed a hospital database schema with Patients, Doctors, and Appointments tables using primary and foreign keys.
- Established relationships by linking appointments with patients and doctors to manage scheduling and interactions.
- Inserted sample data and used JOIN queries to retrieve doctor appointments and patient history.
- Applied aggregate function (COUNT with GROUP BY) to calculate total patients treated by each doctor

Task 3 – Library Database**PROMPT:**

generate a schema for a library management system that include tables books, members and loans

insert sample data into the tables

query to retrieve all books currently issued

query to find overdue books where loan date>30days

query to count number of books loaned by each member

CODE:


```
JS script.js # styles.css <> index.html L1.py Labtest.py labtest2.py lab_15_1.py lab16.db lab16:
lab16.sql
1 --generate a schema for a library management system that include tables books, members and loans
2 CREATE TABLE books1 (
3     book_id INT PRIMARY KEY,
4     title VARCHAR(255) NOT NULL,
5     author VARCHAR(255) NOT NULL,
6     published_date DATE,
7     available_copies INT DEFAULT 0
8 );
9
10 CREATE TABLE members1 (
11     member_id INT PRIMARY KEY,
12     name VARCHAR(255) NOT NULL,
13     email VARCHAR(255) UNIQUE NOT NULL,
14     join_date DATE
15 );
16
17 CREATE TABLE loans1 (
18     loan_id INT PRIMARY KEY,
19     book_id INT,
20     member_id INT,
21     loan_date DATE,
22     return_date DATE,
23     FOREIGN KEY (book_id) REFERENCES books1(book_id),
24     FOREIGN KEY (member_id) REFERENCES members1(member_id)
25 );
26
27 --insert sample values into all tables
28 INSERT INTO books1 (book_id, title, author, published_date, available_copies) VALUES
29 (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
30 (2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
31 (3, '1984', 'George Orwell', '1949-06-08', 5);
32
33 INSERT INTO members1 (member_id, name, email, join_date) VALUES
34 (1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
35 (2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
36
37 INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
```

```
JS script.js # styles.css <> index.html L1.py Labtest.py labtest2.py lab_15_1.py lab16.db lab16:
lab16.sql
29 (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
30 (2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
31 (3, '1984', 'George Orwell', '1949-06-08', 5);
32
33 INSERT INTO members1 (member_id, name, email, join_date) VALUES
34 (1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
35 (2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
36
37 INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
38 (1, 1, 1, '2023-03-01', NULL),
39 (2, 2, 2, '2023-03-05', '2023-03-20'),
40 (3, 3, 1, '2023-03-10', NULL);
41
42
43 --query to retrieve all books currently issued
44 SELECT b.title, m.name, l.loan_date
45 FROM loans1 l
46 JOIN books1 b ON l.book_id = b.book_id
47 JOIN members1 m ON l.member_id = m.member_id
48 WHERE l.return_date IS NULL;
49
50
51 --query to find overdue books where loan date>30days
52 SELECT b.title, m.name, l.loan_date
53 FROM loans1 l
54 JOIN books1 b ON l.book_id = b.book_id
55 JOIN members1 m ON l.member_id = m.member_id
56 WHERE l.return_date IS NULL
57 AND l.loan_date < DATE('now', '-30 days');
58
59 --query to count number of books loaned by each member
60 SELECT m.name, COUNT(l.book_id) AS books_loaned
61 FROM loans1 l
62 JOIN members1 m ON l.member_id = m.member_id
63 GROUP BY m.name;
64
```

OUTPUT:

```
lab16.sql
1  --generate a schema for a library management system that include ta
2  CREATE TABLE books1 (
3      book_id INT PRIMARY KEY,
4      title VARCHAR(255) NOT NULL,
5      author VARCHAR(255) NOT NULL,
6      published_date DATE,
7      available_copies INT DEFAULT 0
8  );
9
10 CREATE TABLE members1 (
11     member_id INT PRIMARY KEY,
12     name VARCHAR(255) NOT NULL,
13     email VARCHAR(255) UNIQUE NOT NULL,
14     join_date DATE
15 );
16
17 CREATE TABLE loans1 (
18     loan_id INT PRIMARY KEY,
19     book_id INT,
20     member_id INT,
21     loan_date DATE,
22     return_date DATE,
23     FOREIGN KEY (book_id) REFERENCES books1(book_id),
24     FOREIGN KEY (member_id) REFERENCES members1(member_id)
25 );
26
27
28
29
30
31
32
33
34
35
```

SQL > 1 / 1 > 1 - 0 of 0

```
CREATE TABLE books1 (
book_id INT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
author VARCHAR(255) NOT NULL,
published_date DATE,
available_copies INT DEFAULT 0
);
```

SQL > 1 / 1 > 1 - 0 of 0

```
CREATE TABLE members1 (
member_id INT PRIMARY KEY,
name VARCHAR(255) NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
join_date DATE
);
```

SQL > 1 / 1 > 1 - 0 of 0

```
CREATE TABLE loans1 (
loan_id INT PRIMARY KEY,
book_id INT,
member_id INT,
loan_date DATE,
return_date DATE,
FOREIGN KEY (book_id) REFERENCES books1(book_id),
FOREIGN KEY (member_id) REFERENCES members1(member_id)
);
```

```
lab16.sql
10 CREATE TABLE members1 (
11     member_id INT PRIMARY KEY,
12     name VARCHAR(255) NOT NULL,
13     email VARCHAR(255) UNIQUE NOT NULL,
14     join_date DATE
15 );
16
17 CREATE TABLE loans1 (
18     loan_id INT PRIMARY KEY,
19     book_id INT,
20     member_id INT,
21     loan_date DATE,
22     return_date DATE,
23     FOREIGN KEY (book_id) REFERENCES books1(book_id),
24     FOREIGN KEY (member_id) REFERENCES members1(member_id)
25 );
26
27 --insert sample values into all tables
28 INSERT INTO books1 (book_id, title, author, published_date, available_copies) VALUES
29 (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
30 (2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
31 (3, '1984', 'George Orwell', '1949-06-08', 5);
32
33 INSERT INTO members1 (member_id, name, email, join_date) VALUES
34 (1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
35 (2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
36
37 INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
38 (1, 1, 1, '2023-03-01', NULL),
39 (2, 2, 2, '2023-03-05', '2023-03-20'),
40 (3, 3, 1, '2023-03-10', NULL);
```

SQL > 1 / 1 > 1 - 0 of 0

```
INSERT INTO books1 (book_id, title, author, published_date, available_copies) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
(3, '1984', 'George Orwell', '1949-06-08', 5);
```

SQL > 1 / 1 > 1 - 0 of 0

```
INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
```

SQL > 1 / 1 > 1 - 0 of 0

```
INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);
```

```
lab16.sql
7  --insert sample values into all tables
8  INSERT INTO books1 (book_id, title, author, published_date, available_copies) VALUES
9  (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
10 (2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2),
11 (3, '1984', 'George Orwell', '1949-06-08', 5);
12
13 INSERT INTO members1 (member_id, name, email, join_date) VALUES
14 (1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
15 (2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
16
17 INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
18 (1, 1, 1, '2023-03-01', NULL),
19 (2, 2, 2, '2023-03-05', '2023-03-20'),
20 (3, 3, 1, '2023-03-10', NULL);
21
22
23
24
```

SQL > 1 / 1 > 1 - 0 of 0

```
INSERT INTO members1 (member_id, name, email, join_date) VALUES
(1, 'Alice Smith', 'alice.smith@example.com', '2023-01-15'),
(2, 'Bob Johnson', 'bob.johnson@example.com', '2023-02-20');
```

SQL > 1 / 1 > 1 - 0 of 0

```
INSERT INTO loans1 (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2023-03-01', NULL),
(2, 2, 2, '2023-03-05', '2023-03-20'),
(3, 3, 1, '2023-03-10', NULL);
```

```
lab16.sql
1 (1, 1, 1, '2023-03-01', NULL),
2 (2, 2, 2, '2023-03-05', '2023-03-20'),
3 (3, 3, 1, '2023-03-10', NULL);
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
--query to retrieve all books currently issued
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

```
SQL
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

title	name	loan_date
The Great Gatsby	Alice Smith	2023-03-01
1984	Alice Smith	2023-03-10

```
lab16.sql
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
--query to find overdue books where loan date>30days
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < DATE('now', '-30 days');
```

```
SQL
SELECT b.title, m.name, l.loan_date
FROM loans1 l
JOIN books1 b ON l.book_id = b.book_id
JOIN members1 m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < DATE('now', '-30 days');
```

title	name	loan_date
The Great Gatsby	Alice Smith	2023-03-01
1984	Alice Smith	2023-03-10

```
lab16.sql
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
--query to count number of books loaned by each member
SELECT m.name, COUNT(l.book_id) AS books_loaned
FROM loans1 l
JOIN members1 m ON l.member_id = m.member_id
GROUP BY m.name;
```

```
SQL
SELECT m.name, COUNT(l.book_id) AS books_loaned
FROM loans1 l
JOIN members1 m ON l.member_id = m.member_id
GROUP BY m.name;
```

name	books_loaned
Alice Smith	2
Bob Johnson	1

EXPLANATION:

- Designed a library database with Books, Members, and Loans tables using primary and foreign keys to manage relationships.
- Inserted sample data and used JOIN queries to retrieve issued books and track member borrowing activity.
- Identified overdue books using date conditions (DATE('now', '-30 days')) for time-based filtering.
- Applied aggregate function (COUNT with GROUP BY) to calculate the number of books borrowed by each member.

Task 4 – Real-Time Application: Online Shopping Database

PROMPT:

--generate a scheme for real time application online shopping database that include tables

-- users products orders and orderdetails in sqlite syntax

insert sample data into all tables

query to retrieve all orders by a user in sqlite synta

query to find the most purchased product

-query to calculate total revenue in a given month using sqlite syntax

CODE:

```
1  --generate a scheme for real time application online shopping database that include tables
2  -- users products orders and orderdetails in sqlite syntax
3  CREATE TABLE users (
4      id INTEGER PRIMARY KEY AUTOINCREMENT,
5      username TEXT NOT NULL UNIQUE,
6      email TEXT NOT NULL UNIQUE,
7      password TEXT NOT NULL,
8      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9  );
10
11 CREATE TABLE products (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     name TEXT NOT NULL,
14     description TEXT,
15     price REAL NOT NULL,
16     stock INTEGER NOT NULL,
17     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
18 );
19
20 CREATE TABLE orders (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     user_id INTEGER NOT NULL,
23     total_amount REAL NOT NULL,
24     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
25     FOREIGN KEY (user_id) REFERENCES users(id)
26 );
27
28 CREATE TABLE order_details (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     order_id INTEGER NOT NULL,
31     product_id INTEGER NOT NULL,
32     quantity INTEGER NOT NULL,
33     price REAL NOT NULL,
34     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
35     FOREIGN KEY (order_id) REFERENCES orders(id),
36     FOREIGN KEY (product_id) REFERENCES products(id)
37 );
```

```

script.js  styles.css  index.html  L1.py  Labtest.py  labtest2.py  lab_15_1.py  lab16.db  la
lab16.sql
38
39
40 --insert sample data into all tables
41 INSERT INTO users (username, email, password) VALUES
42 ('john_doe', 'john.doe@example.com', 'hashed_password_1'),
43 ('jane_smith', 'jane.smith@example.com', 'hashed_password_2');
44
45 INSERT INTO products (name, description, price, stock) VALUES
46 ('Laptop', 'A high-performance laptop', 999.99, 10),
47 ('Smartphone', 'A latest model smartphone', 499.99, 20);
48
49 INSERT INTO orders (user_id, total_amount) VALUES
50 (1, 1499.98),
51 (2, 499.99);
52
53 INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
54 (1, 1, 1, 999.99),
55 (1, 2, 1, 499.99),
56 (2, 2, 1, 499.99);
57
58 --query to retrieve all orders by a user in sqlite syntax
59 select o.id, o.total_amount, o.created_at
60 from orders o
61 join users u on o.user_id = u.id
62 where u.username = 'john_doe';
63
64
65
66
67 --query to find the most purchased product
68 SELECT p.name, SUM(od.quantity) AS total_quantity
69 FROM products p
70 JOIN order_details od ON p.id = od.product_id
71 GROUP BY p.id
72 ORDER BY total_quantity DESC
73 LIMIT 1;
74
75
76
77 -- --query to calculate total revenue in a given month using sqlite syntax
78 SELECT strftime('%Y-%m', created_at) AS month, SUM(total_amount) AS total_revenue
79 FROM orders
80 WHERE created_at >= '2024-01-01' AND created_at < '2024-02-01'
81 GROUP BY month;
82
83
84

```

OUTPUT:

```

1 --generate a scheme for real time application online shopping datab
2 -- users products orders and orderdetails in sqlite syntax
3 CREATE TABLE users (
4     id INTEGER PRIMARY KEY AUTOINCREMENT,
5     username TEXT NOT NULL UNIQUE,
6     email TEXT NOT NULL UNIQUE,
7     password TEXT NOT NULL,
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE TABLE products (
12     id INTEGER PRIMARY KEY AUTOINCREMENT,
13     name TEXT NOT NULL,
14     description TEXT,
15     price REAL NOT NULL,
16     stock INTEGER NOT NULL,
17     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
18 );
19
20 CREATE TABLE orders (
21     id INTEGER PRIMARY KEY AUTOINCREMENT,
22     user_id INTEGER NOT NULL,
23     total_amount REAL NOT NULL,
24     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
25     FOREIGN KEY (user_id) REFERENCES users(id)
26 );
27
28 CREATE TABLE order_details (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     order_id INTEGER NOT NULL,
31     product_id INTEGER NOT NULL,
32     quantity INTEGER NOT NULL,
33     price REAL NOT NULL,
34     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
35     FOREIGN KEY (order_id) REFERENCES orders(id),
36     FOREIGN KEY (product_id) REFERENCES products(id)
37 );

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  description TEXT,
  price REAL NOT NULL,
  stock INTEGER NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
CREATE TABLE orders (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  total_amount REAL NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
CREATE TABLE order_details (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,

```

```

28 CREATE TABLE order_details (
29     id INTEGER PRIMARY KEY AUTOINCREMENT,
30     order_id INTEGER NOT NULL,
31     product_id INTEGER NOT NULL,
32     quantity INTEGER NOT NULL,
33     price REAL NOT NULL,
34     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
35     FOREIGN KEY (order_id) REFERENCES orders(id),
36     FOREIGN KEY (product_id) REFERENCES products(id)
37 );
38
39
40 --insert sample data into all tables
41 INSERT INTO users (username, email, password) VALUES
42 ('john_doe', 'john.doe@example.com', 'hashed_password_1'),
43 ('jane_smith', 'jane.smith@example.com', 'hashed_password_2');
44
45 INSERT INTO products (name, description, price, stock) VALUES
46 ('Laptop', 'A high-performance laptop', 999.99, 10),
47 ('Smartphone', 'A latest model smartphone', 499.99, 20);
48
49 INSERT INTO orders (user_id, total_amount) VALUES
50 (1, 1499.98),
51 (2, 499.99);
52
53 INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
54 (1, 1, 1, 999.99),
55 (1, 2, 1, 499.99),
56 (2, 2, 1, 499.99);
57
58
59
60
61
62
63 --query to retrieve all orders by a user

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
INSERT INTO users (username, email, password) VALUES
('john_doe', 'john.doe@example.com', 'hashed_password_1'),
('jane_smith', 'jane.smith@example.com', 'hashed_password_2');

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
INSERT INTO products (name, description, price, stock) VALUES
('Laptop', 'A high-performance laptop', 999.99, 10),
('Smartphone', 'A latest model smartphone', 499.99, 20);

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
INSERT INTO orders (user_id, total_amount) VALUES
(1, 1499.98),
(2, 499.99);

```

```

SQL ▶ < 1 / 1 > 1 - 0 of 0
INSERT INTO order_details (order_id, product_id, quantity, price) VALUES
(1, 1, 1, 999.99),
(1, 2, 1, 499.99),
(2, 2, 1, 499.99);

```

```

1 -- (1, 1, 1, 999.99),
2 -- (1, 2, 1, 499.99),
3 -- (2, 2, 1, 499.99);
4
5 --query to retrieve all orders by a user in sqlite syntax
6 select o.id, o.total_amount, o.created_at
7 from orders o
8 join users u on o.user_id = u.id
9 where u.username = 'john_doe';
10
11

```

```

SQL ▶ < 1 / 1 > 1 - 1 of 1

```

id	total_amount	created_at
1	1499.98	2026-03-24 18:01:50

```
lab16.sql
61 join users u on o.user_id = u.id
62 where u.username = 'john_doe';
63
64
65
66
67 --query to find the most purchased product
68 SELECT p.name, SUM(od.quantity) AS total_quantity
69 FROM products p
70 JOIN order_details od ON p.id = od.product_id
71 GROUP BY p.id
72 ORDER BY total_quantity DESC
73 LIMIT 1;
74
```

name	total_quantity
Smartphone	2

EXPLANATION:

- Designed an online shopping database with Users, Products, Orders, and Order Details tables using primary and foreign keys to maintain relationships.
- Inserted sample data and used JOIN queries to retrieve user orders and identify the most purchased product.
- Applied aggregate functions like SUM and GROUP BY to calculate total quantity and revenue.
- Used SQLite date function (strftime) to filter and compute monthly revenue efficiently.

Task 5 – University Examination Database

PROMPT:

design a database schema for a university examination system with tables for students , subjects ,exams and results also define primary keys,foreign keys and referential integrity constraints

insert sample data into the tables

query to insert examination results for a student

query to Retrieve subject-wise marks for a student

query to Calculate average marks for each subject

CODE:

```
lab16.sql
1  --design a database schema for a university examination system
2  --with tables for students , subjects ,exams and results
3  --also define primary keys,foreign keys and referential integrity constraints.
4  CREATE TABLE students1 (
5      student_id INT PRIMARY KEY,
6      first_name VARCHAR(50) NOT NULL,
7      last_name VARCHAR(50) NOT NULL,
8      email VARCHAR(100) UNIQUE NOT NULL
9  );
10 CREATE TABLE subjects (
11     subject_id INT PRIMARY KEY,
12     subject_name VARCHAR(100) NOT NULL
13 );
14 CREATE TABLE exams (
15     exam_id INT PRIMARY KEY,
16     subject_id INT,
17     exam_date DATE NOT NULL,
18     FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
19 );
20 CREATE TABLE results (
21     result_id INT PRIMARY KEY,
22     student_id INT,
23     exam_id INT,
24     score DECIMAL(5,2) NOT NULL,
25     FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
26     FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCADE
27 );
28
29 --insert sample data into the tables
30 INSERT INTO students1 (student_id, first_name, last_name, email) VALUES
31 (1, 'John', 'Doe', 'john.doe@university.edu'),
32 (2, 'Jane', 'Smith', 'jane.smith@university.edu');
33
34 INSERT INTO subjects (subject_id, subject_name) VALUES
35 (1, 'Mathematics'),
36 (2, 'Physics');
37
38 INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
39 (1, 1, '2024-12-01'),
40 (2, 2, '2024-12-02');
41
42 INSERT INTO results (result_id, student_id, exam_id, score) VALUES
43 (1, 1, 1, 85.50),
44 (2, 1, 2, 90.00),
45 (3, 2, 1, 78.00),
46 (4, 2, 2, 88.50);
47
48
49 --query to insert examination results for a student
50 INSERT INTO results (result_id, student_id, exam_id, score) VALUES
51 (5, 1, 1, 92.00);
52
53
54 --query to Retrieve subject-wise marks for a student
55 SELECT s.first_name, s.last_name, sub.subject_name, r.score
56 FROM students1 s
57 JOIN results r ON s.student_id = r.student_id
58 JOIN exams e ON r.exam_id = e.exam_id
59 JOIN subjects sub ON e.subject_id = sub.subject_id
60 WHERE s.student_id = 1;
61
62 --query to Calculate average marks for each subject
63 SELECT sub.subject_name, AVG(r.score) AS average_score
64 FROM subjects sub
65 JOIN exams e ON sub.subject_id = e.subject_id
66 JOIN results r ON e.exam_id = r.exam_id
67 GROUP BY sub.subject_name;
68
```

OUTPUT:


```
labtest.py X labtest2.py lab_15_1.py lab16.db lab16.sql X ... SQLite X ...
lab16.sql
1 --design a database schema for a university examination system
2 --with tables for students , subjects ,exams and results
3 --also define primary keys,foreign keys and referential integrity constraints
4 CREATE TABLE students1 (
5     student_id INT PRIMARY KEY,
6     first_name VARCHAR(50) NOT NULL,
7     last_name VARCHAR(50) NOT NULL,
8     email VARCHAR(100) UNIQUE NOT NULL
9 );
10 Create TABLE subjects (
11     subject_id INT PRIMARY KEY,
12     subject_name VARCHAR(100) NOT NULL
13 );
14 CREATE TABLE exams (
15     exam_id INT PRIMARY KEY,
16     subject_id INT,
17     exam_date DATE NOT NULL,
18     FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
19 );
20 CREATE TABLE results (
21     result_id INT PRIMARY KEY,
22     student_id INT,
23     exam_id INT,
24     score DECIMAL(5,2) NOT NULL,
25     FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE
26     FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCADE
27 );
28
29
30
```

```
SQLite
SQL < 1 / 1 > 1 - 0 of 0
CREATE TABLE students1 (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);

SQL < 1 / 1 > 1 - 0 of 0
Create TABLE subjects (
  subject_id INT PRIMARY KEY,
  subject_name VARCHAR(100) NOT NULL
);

SQL < 1 / 1 > 1 - 0 of 0
CREATE TABLE exams (
  exam_id INT PRIMARY KEY,
  subject_id INT,
  exam_date DATE NOT NULL,
  FOREIGN KEY (subject_id) REFERENCES subjects(subject_id) ON DELETE CASCADE
);

SQL < 1 / 1 > 1 - 0 of 0
CREATE TABLE results (
  result_id INT PRIMARY KEY,
  student_id INT,
  exam_id INT,
  score DECIMAL(5,2) NOT NULL,
  FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
  FOREIGN KEY (exam_id) REFERENCES exams(exam_id) ON DELETE CASCADE
);
```

```
labtest.py X labtest2.py lab_15_1.py lab16.db lab16.sql X ... SQLite X ...
lab16.sql
28
29 --insert sample data into the tables
30 INSERT INTO students1 (student_id, first_name, last_name, email) VALUES
31 (1, 'John', 'Doe', 'john.doe@university.edu'),
32 (2, 'Jane', 'Smith', 'jane.smith@university.edu');
33
34 INSERT INTO subjects (subject_id, subject_name) VALUES
35 (1, 'Mathematics'),
36 (2, 'Physics');
37
38 INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
39 (1, 1, '2024-12-01'),
40 (2, 2, '2024-12-02');
41
42 INSERT INTO results (result_id, student_id, exam_id, score) VALUES
43 (1, 1, 1, 85.50),
44 (2, 1, 2, 90.00),
45 (3, 2, 1, 78.00),
46 (4, 2, 2, 88.50);
47
48
```

```
SQLite
SQL < 1 / 1 > 1 - 0 of 0
INSERT INTO students1 (student_id, first_name, last_name, email) VALUES
(1, 'John', 'Doe', 'john.doe@university.edu'),
(2, 'Jane', 'Smith', 'jane.smith@university.edu');

SQL < 1 / 1 > 1 - 0 of 0
INSERT INTO subjects (subject_id, subject_name) VALUES
(1, 'Mathematics'),
(2, 'Physics');

SQL < 1 / 1 > 1 - 0 of 0
INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
(1, 1, '2024-12-01'),
(2, 2, '2024-12-02');

SQL < 1 / 1 > 1 - 0 of 0
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(1, 1, 1, 85.50),
(2, 1, 2, 90.00),
(3, 2, 1, 78.00),
(4, 2, 2, 88.50);
```

```
labtest.py X labtest2.py lab_15_1.py lab16.db lab16.sql X ... SQLite X ...
lab16.sql
45 (3, 2, 1, 78.00),
46 (4, 2, 2, 88.50);
47
48
49 --query to insert examination results for a student
50 INSERT INTO results (result_id, student_id, exam_id, score) VALUES
51 (5, 1, 1, 92.00);
52
```

```
SQLite
SQL < 1 / 1 > 1 - 0 of 0
INSERT INTO results (result_id, student_id, exam_id, score) VALUES
(5, 1, 1, 92.00);
```

The screenshot displays a SQLite IDE with two panels. The top panel shows a SQL script with an insert statement and a join query. The bottom panel shows an aggregate query and its results.

Top Panel SQL Script:

```
46 (4, 2, 2, 88.50);
47
48
49 --query to insert examination results for a student
50 INSERT INTO results (result_id, student_id, exam_id, score) VALUES
51 (5, 1, 1, 92.00);
52
53
54 --query to Retrieve subject-wise marks for a student
55 SELECT s.first_name, s.last_name, sub.subject_name, r.score
56 FROM students1 s
57 JOIN results r ON s.student_id = r.student_id
58 JOIN exams e ON r.exam_id = e.exam_id
59 JOIN subjects sub ON e.subject_id = sub.subject_id
60 WHERE s.student_id = 1;
```

Top Panel SQL Query and Results:

```
SQL ▾ < 1 / 1 > 1 - 3 of 3
SELECT s.first_name, s.last_name, sub.subject_name, r.score
FROM students1 s
JOIN results r ON s.student_id = r.student_id
JOIN exams e ON r.exam_id = e.exam_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE s.student_id = 1;
```

first_name	last_name	subject_name	score
John	Doe	Mathematics	85.5
John	Doe	Physics	90
John	Doe	Mathematics	92

Bottom Panel SQL Script:

```
62 --query to calculate average marks for each subject
63 SELECT sub.subject_name, AVG(r.score) AS average_score
64 FROM subjects sub
65 JOIN exams e ON sub.subject_id = e.subject_id
66 JOIN results r ON e.exam_id = r.exam_id
67 GROUP BY sub.subject_name;
68
```

Bottom Panel SQL Query and Results:

```
SQL ▾ < 1 / 1 > 1 - 2 of 2
SELECT sub.subject_name, AVG(r.score) AS average_score
FROM subjects sub
JOIN exams e ON sub.subject_id = e.subject_id
JOIN results r ON e.exam_id = r.exam_id
GROUP BY sub.subject_name;
```

subject_name	average_score
Mathematics	85.16666666666667
Physics	89.25

EXPLANATION:

- Designed a university examination database with Students, Subjects, Exams, and Results tables using primary and foreign keys to maintain relationships.
- Ensured referential integrity by linking results with students and exams, and exams with subjects.
- Inserted sample data and used JOIN queries to retrieve subject-wise marks for a student.
- Applied aggregate function (AVG with GROUP BY) to calculate average marks for each subject.